

PROJEKT

System zarządzania danymi giełdowymi — testy bazy danych SQL

Cel projektu:

Przetestowanie integralności, spójności i jakości danych w systemie gromadzącym informacje o rynkach giełdowych, spółkach, notowaniach dziennych i transakcjach.

Narzędzie:

MySQL / phpMyAdmin

Zakres testów:

Walidacja danych, relacje między tabelami, logika biznesowa, testy jakości danych, testy CRUD.

TEST CASE 1

Tytuł: Weryfikacja powiązań między spółkami a rynkami

Cel: Sprawdzić, czy każda spółka przypisana jest do istniejącego rynku.

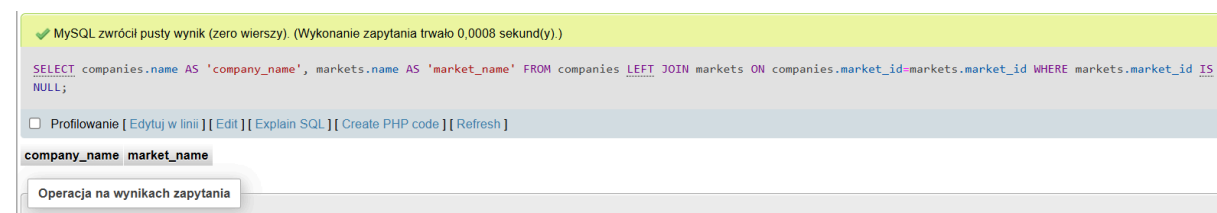
Opis: Upewnij się, że w tabeli *companies* nie występują rekordy, których *market_id* nie istnieje w tabeli *markets*.

✅ **Test zaliczony**

Rzeczywisty wynik: Zapytanie nie zwróciło żadnych rekordów niespełniających warunku. Wszystkie rekordy w tabeli *companies* posiadają poprawne wartości *market_id* zgodne z tabelą *markets*.

Oczekiwany wynik: Wszystkie spółki mają przypisany poprawny rynek.

```
SELECT companies.name AS 'company_name', markets.name AS 'market_name'
FROM companies
LEFT JOIN markets ON companies.market_id=markets.market_id
WHERE markets.market_id IS NULL;
```



TEST CASE 2

Tytuł: Weryfikacja unikalności tickerów spółek

Cel: Sprawdzić, czy symbole giełdowe (tickery) są unikalne.

Opis: Zidentyfikuj ewentualne duplikaty w kolumnie *ticker* w tabeli *companies*.

✅ **Test zaliczony**

Rzeczywisty wynik: Zapytanie nie zwróciło żadnych rekordów – wszystkie tickery występują tylko raz, co potwierdza ich unikalność.

Oczekiwany wynik: Brak duplikatów tickerów.

```
SELECT ticker, COUNT(*) AS "liczba_wystąpień"
FROM `companies`
GROUP BY ticker
HAVING COUNT(*) > 1;
```

✓ MySQL zwrócił pusty wynik (zero wierszy). (Wykonanie zapytania trwało 0,0004 sekund(y).)

```
SELECT ticker, COUNT(*) AS "liczba_wystąpień" FROM `companies` GROUP BY ticker HAVING COUNT(*) > 1;
```

☐ Profilowanie [[Edytuj w linii](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

ticker **liczba_wystąpień**

Operacja na wynikach zapytania

TEST CASE 3

Tytuł: Sprawdzenie poprawności dat w notowaniach

Cel: Zweryfikować, że daty notowań (*trade_date*) nie wykraczają poza bieżącą datę.

Opis: Ustal, czy w tabeli *dailyquotes* istnieją rekordy z datą w przyszłości.

✅ **Test zaliczony**

Rzeczywisty wynik: Zapytanie nie zwróciło żadnych rekordów – wszystkie daty notowań są równe lub wcześniejsze niż bieżąca data.

Oczekiwany wynik: Wszystkie notowania mają daty nieprzekraczające bieżącego dnia.

```
SELECT *
FROM dailyquotes
WHERE trade_date > CURRENT_DATE;
```

✓ MySQL zwrócił pusty wynik (zero wierszy). (Wykonanie zapytania trwało 0,0014 sekund(y).)

```
SELECT * FROM dailyquotes WHERE trade_date > CURRENT_DATE;
```

☐ Profilowanie [Edytuj w linii] [Edit] [Explain SQL] [Create PHP code] [Refresh]

quote_id	company_id	trade_date	open_price	close_price	high_price	low_price	volume
----------	------------	------------	------------	-------------	------------	-----------	--------

Operacja na wynikach zapytania

TEST CASE 4

Tytuł: Walidacja zakresów cenowych w notowaniach

Cel: Upewnić się, że wartości *high_price*, *low_price*, *open_price* i *close_price* są logicznie poprawne.

Opis: Sprawdź, czy $high_price \geq low_price$, a wszystkie ceny są dodatnie.

✓ **Test zaliczony**

Rzeczywisty wynik: Zapytanie nie zwróciło żadnych rekordów — wszystkie wartości cenowe są dodatnie i spełniają zależność $high_price \geq low_price$.

Oczekiwany wynik: Brak niepoprawnych wartości cenowych.

```
SELECT high_price, low_price, open_price, close_price
FROM dailyquotes
WHERE high_price < low_price
      OR high_price < 0
      OR low_price < 0
      OR open_price < 0
      OR close_price < 0;
```

✓ MySQL zwrócił pusty wynik (zero wierszy). (Wykonanie zapytania trwało 0,0070 sekund(y).)

```
SELECT high_price, low_price, open_price, close_price FROM dailyquotes WHERE high_price < low_price OR high_price < 0 OR low_price < 0 OR open_price < 0 OR close_price < 0;
```

☐ Profilowanie [Edytuj w linii] [Edit] [Explain SQL] [Create PHP code] [Refresh]

high_price	low_price	open_price	close_price
------------	-----------	------------	-------------

Operacja na wynikach zapytania

TEST CASE 5

Tytuł: Weryfikacja kompletności danych notowań

Cel: Sprawdzić, czy dla każdej spółki istnieją dane notowań.

Opis: Ustal, czy w tabeli *companies* istnieją spółki, które nie mają powiązanych rekordów w tabeli *dailyquotes*.

✓ **Test zaliczony**

Rzeczywisty wynik: Zapytanie nie zwróciło żadnych rekordów — każda spółka posiada co najmniej jedno notowanie w tabeli *dailyquotes*.

Oczekiwany wynik: Każda spółka ma co najmniej jedno notowanie.

```
SELECT companies.name, dailyquotes.*
FROM companies
LEFT JOIN dailyquotes ON companies.company_id = dailyquotes.company_id
WHERE dailyquotes.company_id IS NULL;
```

✓ MySQL zwrócił pusty wynik (zero wierszy). (Wykonanie zapytania trwało 0,0005 sekund(y).)

SELECT companies.name, dailyquotes.* FROM companies LEFT JOIN dailyquotes ON companies.company_id = dailyquotes.company_id WHERE dailyquotes.company_id IS NULL;

☐ Profilowanie [Edytuj w linii] [Edit] [Explain SQL] [Create PHP code] [Refresh]

name quote_id company_id trade_date open_price close_price high_price low_price volume

Operacja na wynikach zapytania

TEST CASE 6

Tytuł: Weryfikacja poprawności transakcji

Cel: Sprawdzić, czy wszystkie transakcje (*trades*) dotyczą istniejących spółek i dat z notowaniami.

Opis: Ustal, czy istnieją transakcje, które nie mają odpowiadającego notowania w *dailyquotes* dla tej samej spółki i daty.

✓ **Test zaliczony**

Rzeczywisty wynik: Zapytanie nie zwróciło żadnych rekordów — wszystkie transakcje mają odpowiadające notowania w tabeli *dailyquotes* dla tej samej spółki i daty.

Oczekiwany wynik: Wszystkie transakcje powiązane z prawidłowymi notowaniami.

```
SELECT DISTINCT trades.trade_id, companies.name, dailyquotes.trade_date
FROM trades
LEFT JOIN companies ON trades.company_id = companies.company_id
LEFT JOIN dailyquotes ON trades.company_id = dailyquotes.company_id
WHERE companies.company_id IS NULL
OR dailyquotes.company_id IS NULL;
```

✓ MySQL zwrócił pusty wynik (zero wierszy). (Wykonanie zapytania trwało 0,0068 sekund(y).)

SELECT DISTINCT trades.trade_id, companies.name, dailyquotes.trade_date FROM trades LEFT JOIN companies ON trades.company_id=companies.company_id LEFT JOIN dailyquotes ON trades.company_id=dailyquotes.company_id WHERE companies.company_id IS NULL OR dailyquotes.company_id IS NULL;

☐ Profilowanie [Edytuj w linii] [Edit] [Explain SQL] [Create PHP code] [Refresh]

trade_id name trade_date

Operacja na wynikach zapytania

TEST CASE 7

Tytuł: Sprawdzenie integralności danych użytkowników i subskrypcji

Cel: Upewnić się, że każda subskrypcja dotyczy istniejącego użytkownika i rynku.

Opis: Zidentyfikuj rekordy w *subscriptions*, które odnoszą się do nieistniejących *user_id* lub *market_id*.

✓ **Test zaliczony**

Rzeczywisty wynik:

Oczekiwany wynik: Brak błędnych powiązań subskrypcji.

```
SELECT subscriptions.subscription_id, users.user_id, markets.market_id
FROM subscriptions
LEFT JOIN users ON subscriptions.user_id=users.user_id
LEFT JOIN markets ON subscriptions.market_id=markets.market_id
WHERE users.user_id IS NULL OR markets.market_id IS NULL;
```

✓ MySQL zwrócił pusty wynik (zero wierszy). (Wykonanie zapytania trwało 0,0005 sekund(y).)

[SELECT](#) [subscriptions.subscription_id](#), [users.user_id](#), [markets.market_id](#) [FROM](#) [subscriptions](#) [LEFT JOIN](#) [users](#) [ON](#) [subscriptions.user_id=users.user_id](#) [LEFT JOIN](#) [markets](#) [ON](#) [subscriptions.market_id=markets.market_id](#) [WHERE](#) [users.user_id IS NULL](#) [OR](#) [markets.market_id IS NULL](#);

☐ Profilowanie [[Edytuj w linii](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

[subscription_id](#) [user_id](#) [market_id](#)

Operacja na wynikach zapytania

TEST CASE 8

Tytuł: Walidacja aktywnych subskrypcji

Cel: Zweryfikować, ilu użytkowników ma aktywną subskrypcję (bez daty zakończenia lub z datą w przyszłości).

Opis: Policz aktywne subskrypcje na każdy rynek.

✓ **Test zaliczony**

Rzeczywisty wynik: Zapytanie zwróciło listę wszystkich rynków wraz z liczbą użytkowników posiadających aktywną subskrypcję. Wszystkie dane są zgodne z oczekiwaniem.

Oczekiwany wynik: Lista rynków i liczba aktywnych użytkowników.

```
SELECT COUNT(subscriptions.user_id) AS 'active subscriptions', markets.name
FROM subscriptions
INNER JOIN markets ON subscriptions.market_id = markets.market_id
WHERE end_date > CURRENT_DATE OR end_date IS NULL
GROUP BY markets.name;
```

✓ Pokazano wiersze 0 - 2 (3 total, Wykonanie zapytania trwało 0,0072 sekund(y).)

```
SELECT COUNT(subscriptions.user_id) AS 'active subscriptions', markets.name FROM subscriptions INNER JOIN markets ON subscriptions.market_id = markets.market_id WHERE end_date > CURRENT_DATE OR end_date IS NULL GROUP BY markets.name;
```

☐ Profilowanie [Edytuj w linii] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Liczba wierszy: 25 | Filter rows: Przeszukaj tę tabelę

Extra options

active subscriptions	name
1	Euromarket Amsterdam
1	Euromarket Lisbon
1	Euromarket Paris

TEST CASE 9

Tytuł: Weryfikacja poprawności adresów e-mail użytkowników

Cel: Upewnić się, że wszystkie e-maile mają poprawny format.

Opis: Sprawdź, czy w kolumnie *email* występują rekordy bez znaku @ lub kropki.

✓ **Test zaliczony**

Rzeczywisty wynik: Zapytanie zwróciło 0 rekordów – wszystkie adresy e-mail zawierają znak „@” i kropkę.

Oczekiwany wynik: Brak błędnych adresów e-mail.

```
SELECT email
FROM `users`
WHERE email NOT LIKE '%@%' AND email NOT LIKE '%.%';
```

✓ MySQL zwrócił pusty wynik (zero wierszy). (Wykonanie zapytania trwało 0,0006 sekund(y).)

```
SELECT email FROM `users` WHERE email NOT LIKE '%@%' OR email NOT LIKE '%.%';
```

☐ Profilowanie [Edytuj w linii] [Edit] [Explain SQL] [Create PHP code] [Refresh]

email

TEST CASE 10

Tytuł: Sprawdzenie poprawności dat w subskrypcjach

Cel: Zweryfikować, czy data zakończenia (*end_date*) nie jest wcześniejsza niż data rozpoczęcia (*start_date*).

Opis: Znajdź przypadki, w których *end_date* < *start_date*.

✓ **Test zaliczony**

Rzeczywisty wynik: Zapytanie nie zwróciło żadnych rekordów — wszystkie subskrypcje mają poprawny zakres dat.

Oczekiwany wynik: Wszystkie subskrypcje mają logicznie poprawne daty.

```
SELECT start_date, end_date
FROM `subscriptions`
WHERE start_date > end_date;
```

✓ MySQL zwrócił pusty wynik (zero wierszy). (Wykonanie zapytania trwało 0,0003 sekund(y).)

```
SELECT start_date, end_date FROM `subscriptions` WHERE start_date > end_date;
```

☐ Profilowanie [[Edytuj w linii](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

start_date	end_date
------------	----------

TEST CASE 11

Tytuł: Analiza średnich kursów zamknięcia spółek

Cel: Obliczyć średni kurs zamknięcia (*close_price*) dla każdej spółki.

Opis: Sprawdź, które spółki mają najwyższą i najniższą średnią cenę zamknięcia.

✓ **Test zaliczony**

Rzeczywisty wynik: Zapytanie zwróciło listę wszystkich spółek wraz z ich średnim kursem zamknięcia. Dane zostały poprawnie posortowane od najwyższej do najniższej średniej wartości.

Oczekiwany wynik: Raport przedstawiający średni kurs zamknięcia dla każdej spółki, posortowany malejąco.

```
SELECT AVG(dailyquotes.close_price) AS 'avg_close_price', companies.name
FROM dailyquotes
INNER JOIN companies ON dailyquotes.company_id = companies.company_id
GROUP BY companies.name
ORDER BY avg_close_price DESC;
```

✓ Pokazano wiersze 0 - 3 (4 total, Wykonanie zapytania trwało 0,0004 sekund(y).)

```
SELECT AVG(dailyquotes.close_price) AS 'avg_close_price', companies.name FROM dailyquotes INNER JOIN companies ON dailyquotes.company_id = companies.company_id GROUP BY companies.name ORDER BY avg_close_price DESC;
```

☐ Profilowanie [[Edytuj w linii](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Liczba wierszy: 25 | Filter rows:

Extra options

avg_close_price	name
835.500000	LVMH
61.050000	TotalEnergies
19.000000	Philips
13.500000	Galp Energia

TEST CASE 12

Tytuł: Sprawdzenie zgodności cen minimalnych i maksymalnych z logiką giełdową

Cel: Upewnić się, że *close_price* mieści się w zakresie *low_price*–*high_price*.

Opis: Znajdź rekordy w *dailyquotes*, w których *close_price* jest mniejsza od *low_price* lub większa od *high_price*.

✓ **Test zaliczony**

Rzeczywisty wynik: Zapytanie nie zwróciło żadnych rekordów — wszystkie ceny zamknięcia mieszczą się w zakresie ceny minimalnej i maksymalnej.

Oczekiwany wynik: Wszystkie ceny zamknięcia mieszczą się w zakresie ceny minimalnej i maksymalnej.

```
SELECT quote_id, company_id, trade_date, close_price, high_price, low_price
FROM dailyquotes
WHERE close_price > high_price OR close_price < low_price;
```

✓ MySQL zwrócił pusty wynik (zero wierszy). (Wykonanie zapytania trwało 0,0003 sekund(y).)

```
SELECT quote_id, company_id, trade_date, close_price, high_price, low_price FROM dailyquotes WHERE close_price > high_price OR close_price < low_price;
```

☐ Profilowanie [[Edytuj w linii](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

quote_id	company_id	trade_date	close_price	high_price	low_price
----------	------------	------------	-------------	------------	-----------